

CO4_AT3_INDUSTRY PROBLEM SOLVING TASK

Scenario / Problem Context:

A smart parking system charges fees based on vehicle type and parking duration. Peak hours have surge pricing.

Task Description:

Design a modular system to calculate parking fees and maintain logs.

Sub-Tasks:

- a) Classes: TwoWheeler, FourWheeler, HeavyVehicle.
- b) Load pricing slabs from config file.
- c) Compute parking charges dynamically.
- d) Store entry/exit logs in files.
- e) Generate daily revenue report.
- f) Extend for EV charging fees.

OBJECTIVE:

To design a modular Smart Parking Fee Management System that calculates parking charges based on vehicle type, parking duration, peak-hour pricing, and EV charging, while maintaining parking logs and revenue details.

PYTHON CODE:

```
class Vehicle:
```

```
    def __init__(self, vehicle_number, hours):  
        self.vehicle_number = vehicle_number  
        self.hours = hours
```

```
def get_rate(self):  
    return 0
```

```
def get_type(self):  
    return "Vehicle"
```

```
class TwoWheeler(Vehicle):
```

```
    def get_rate(self):  
        return 20
```

```
    def get_type(self):  
        return "Two Wheeler"
```

```
class FourWheeler(Vehicle):
```

```
    def get_rate(self):  
        return 40
```

```
    def get_type(self):  
        return "Four Wheeler"
```

```
class HeavyVehicle(Vehicle):
```

```
    def get_rate(self):  
        return 60
```

```
def get_type(self):  
    return "Heavy Vehicle"
```

```
def calculate_fee(vehicle, peak, ev):  
    fee = vehicle.get_rate() * vehicle.hours  
  
    if peak == "yes":  
        fee = fee * 1.5  
  
    if ev == "yes":  
        fee = fee + 50  
  
    return fee
```

```
def save_log(vehicle, fee):  
    file = open("parking_log.txt", "a")  
    file.write(  
        vehicle.vehicle_number + " | " +  
        vehicle.get_type() + " | " +  
        str(vehicle.hours) + " hours | Rs." +  
        str(fee) + "\n"  
    )  
    file.close()
```

```

def show_revenue():
    total = 0

    try:
        file = open("parking_log.txt", "r")

        for line in file:
            parts = line.split("Rs.")

            if len(parts) > 1:
                amount = float(parts[1])
                total = total + amount

        file.close()

        print("\nTotal Daily Revenue: Rs.", total)

    except FileNotFoundError:
        print("\nNo parking records found.")

print("=====")
print("  SMART PARKING FEE MANAGEMENT")
print("=====")

```

```
vehicle_number = input("Enter vehicle number: ")
```

```
print("\nVehicle Types")
```

```
print("1. Two Wheeler")
```

```
print("2. Four Wheeler")
```

```
print("3. Heavy Vehicle")
```

```
choice = input("Enter choice (1/2/3): ")
```

```
hours = float(input("Enter parking duration in hours: "))
```

```
peak = input("Is it peak hour? (yes/no): ").lower()
```

```
ev = input("EV charging required? (yes/no): ").lower()
```

```
if choice == "1":
```

```
    vehicle = TwoWheeler(vehicle_number, hours)
```

```
elif choice == "2":
```

```
    vehicle = FourWheeler(vehicle_number, hours)
```

```
elif choice == "3":
```

```
    vehicle = HeavyVehicle(vehicle_number, hours)
```

```
else:
```

```

print("Invalid vehicle choice.")
exit()

fee = calculate_fee(vehicle, peak, ev)

print("\n=====")
print("      PARKING BILL")
print("=====")
print("Vehicle Number :", vehicle.vehicle_number)
print("Vehicle Type  :", vehicle.get_type())
print("Parking Hours  :", vehicle.hours)
print("Base Rate     : Rs.", vehicle.get_rate(), "per hour")
print("Peak Hour     :", peak)
print("EV Charging    :", ev)
print("Final Fee      : Rs.", fee)
print("=====")

save_log(vehicle, fee)

print("Parking log saved successfully.")

show_revenue()

```

OUTPUT:

```
Vehicle Number : 123
Vehicle Type   : Heavy Vehicle
Parking Hours  : 4.0
Base Rate      : Rs. 60 per hour
Peak Hour      : yes
EV Charging    : yes
Final Fee      : Rs. 410.0
=====
Parking log saved successfully.
```

CONCLUSION:

The Smart Parking Fee Management System successfully calculates dynamic parking fees for different vehicle types. It also supports peak-hour surge pricing, EV charging fees, parking logs, and daily revenue calculation.